

Dev TEAM ARCHITECTURE GUIDELINES

Last updated by | Muhammad Fardeen Arif Khan | Apr 21, 2026 at 2:44 PM GMT+5

Dev TEAM ARCHITECTURE GUIDELINES

GOAL

SIMPLE to read, EASY to follow, SAFE to scale.

Anyone should be able to add features without breaking others.

FOLDER STRUCTURE (MANDATORY)

```
Assets/
├── _Project/
│   ├── Core/           // Reusable logic (NO gameplay)
│   ├── Systems/        // Independent systems (Crop, Animal, Task, etc)
│   ├── Gameplay/       // World composition (Player, AI, interactions)
│   ├── UI/             // UI logic (screens, panels, controllers)
│   ├── Data/           // ScriptableObjects (configs)
│   ├── Infrastructure/  // Ads, IAP, Analytics, Backend
│   ├── Bootstrap/      // Entry point (service registration)
│   └── Tests/          // Unit / play mode tests
├── Art/
│   ├── Models/         // 3D models
│   ├── Materials/
│   ├── Textures/
│   └── Animations/
├── UI_Art/
│   ├── Sprites/
│   ├── Atlases/
│   └── Fonts/
├── Audio/
│   ├── SFX/
│   └── Music/
├── Prefabs/
│   ├── Gameplay/
│   ├── UI/
│   └── VFX/
├── Scenes/
├── VFX/
└── Plugins/
```

CORE PRINCIPLES (MANDATORY)

1. SINGLE RESPONSIBILITY (SRP)

One class = One job

GOOD:

InputHandler → handles input

CropService → handles crop logic
UIController → handles UI updates

BAD:

PlayerController → input + crop logic + UI + saving

2. SEPARATION OF CONCERNS

Keep responsibilities clearly separated

Input Layer → reads input
Gameplay Layer → decides what to do
Systems Layer → executes logic
UI Layer → displays data

3. DEPEND ON INTERFACES (NOT CONCRETE CLASSES)

Use ICropService instead of CropService

4. OPEN FOR EXTENSION, CLOSED FOR MODIFICATION

Add new behavior WITHOUT modifying existing code

 NON-NEGOTIABLE RULES

- ✗ No system directly references another system
- ✗ No logic inside UI
- ✗ No heavy logic inside MonoBehaviours
- ✗ No hardcoded values

- ✓ Use EventBus for communication
- ✓ Use ScriptableObjects for data
- ✓ Use Services for logic

 LAYER RESPONSIBILITIES (VERY IMPORTANT)**INPUT LAYER**

Reads player input ONLY
Does NOT contain gameplay logic

GAMEPLAY LAYER

Receives input and decides actions
Calls systems (services)

SYSTEMS LAYER

Contains all core game logic
No dependency on Player, UI, or specific objects

UI LAYER

Displays data

Listens to events

Does NOT contain game logic

🔥 CORE SYSTEMS (BASE IMPLEMENTATION)

SERVICE LOCATOR

```
public static class ServiceLocator
{
    private static Dictionary<Type, object> services = new();

    public static void Register<T>(T service)
    {
        services[typeof(T)] = service;
    }

    public static T Get<T>()
    {
        return (T)services[typeof(T)];
    }
}
```



EVENT BUS

```
public static class EventBus
{
    private static Dictionary<Type, Action<object>> events = new();

    public static void Subscribe<T>(Action<T> callback)
    {
        Type type = typeof(T);

        if (!events.ContainsKey(type))
            events[type] = delegate { };

        events[type] += (e) => callback((T)e);
    }

    public static void Publish<T>(T signal)
    {
        Type type = typeof(T);

        if (events.ContainsKey(type))
            events[type].Invoke(signal);
    }
}
```



INTERACTABLE

```
public interface IInteractable
{
    void Interact(GameObject interactor);
}
```



EXAMPLE SYSTEM (CROP)

DATA

```
[CreateAssetMenu(menuName = "Data/Crop")]
public class CropData : ScriptableObject
{
    public GameObject prefab;
    public float growTime;
}
```



SERVICE

```
public interface ICropService
{
    void Plant(CropData data, Vector3 position);
}

public class CropService : ICropService
{
    public void Plant(CropData data, Vector3 position)
    {
        var obj = GameObject.Instantiate(data.prefab, position, Quaternion.identity);
        var crop = obj.GetComponent<Crop>();
        crop.Init(data);
    }
}
```



CROP MONOBHAVIOUR

```

public class Crop : MonoBehaviour, IInteractable
{
    private CropData data;
    private float timer;
    private bool isReady;

    public void Init(CropData data)
    {
        this.data = data;
        timer = 0;
        isReady = false;
    }

    private void Update()
    {
        if (isReady) return;

        timer += Time.deltaTime;

        if (timer >= data.growTime)
        {
            isReady = true;
        }
    }

    public void Interact(GameObject interactor)
    {
        if (!isReady) return;

        EventBus.Publish(new CropHarvestedSignal());
        Destroy(gameObject);
    }
}

```



EVENT

```

public struct CropHarvestedSignal { }

```



GAMEPLAY EXAMPLE (INPUT + PLAYER)

INPUT HANDLER

```

public class InputHandler : MonoBehaviour
{
    public Vector2 Movement { get; private set; }
    public bool InteractPressed { get; private set; }

    private void Update()
    {
        Movement = new Vector2(Input.GetAxis("Horizontal"), Input.GetAxis("Vertical"));
        InteractPressed = Input.GetKeyDown(KeyCode.E);
    }
}

```



PLAYER CONTROLLER

```
public class PlayerController : MonoBehaviour
{
    [SerializeField] private InputHandler input;

    private ICropService cropService;

    private void Start()
    {
        cropService = ServiceLocator.Get<ICropService>();
    }

    private void Update()
    {
        HandleMovement();
        HandleInteraction();
    }

    private void HandleMovement()
    {
        Vector2 move = input.Movement;
        // Movement logic here
    }

    private void HandleInteraction()
    {
        if (!input.InteractPressed) return;

        // Example interaction logic
        // Detect IInteractable and call Interact()
    }

    public void Plant(CropData data, Vector3 position)
    {
        cropService.Plant(data, position);
    }
}
```



BOOTSTRAP

```
public class GameBootstrapper : MonoBehaviour
{
    private void Awake()
    {
        ServiceLocator.Register<ICropService>(new CropService());
    }
}
```



TEAM CHECKLIST (READ BEFORE COMMIT)

- ✓ Class has ONLY one responsibility
- ✓ Input is handled in InputHandler (NOT PlayerController)
- ✓ No cross-system dependency

- ✓ Uses EventBus if communication needed
- ✓ Uses ScriptableObject for data
- ✓ MonoBehaviour is lightweight
- ✓ Code is easy to read in < 30 seconds

END OF DOCUMENT